

GESP7-速查手册-v1.0

GESP 七级 · D册 速查手册

版本: v1.0 | 日期: 2026-05-01

一、数学与数论

1.1 GCD / LCM

代码块

```
1  #include <numeric>
2  int g = std::gcd(a, b);           // C++17
3  int l = std::lcm(a, b);           // C++17, lcm = a / gcd * b
```

1.2 快速幂

代码块

```
1  long long qpow(long long a, long long b, long long m) {
2      long long res = 1 % m;
3      a %= m;
4      while (b) {
5          if (b & 1) res = res * a % m;
6          a = a * a % m;
7          b >>= 1;
8      }
9      return res;
10 }
11 // O(log b)
```

1.3 质数筛（欧拉筛/线性筛）

代码块

```
1  vector<int> primes;
2  bool vis[N];
3  void sieve(int n) {
4      for (int i = 2; i <= n; ++i) {
```

```

5         if (!vis[i]) primes.push_back(i);
6         for (int p : primes) {
7             if (1LL * i * p > n) break;
8             vis[i * p] = true;
9             if (i % p == 0) break;
10        }
11    }
12 }
13 // O(n)

```

1.4 欧拉函数

代码块

```

1  int phi(int n) {
2      int res = n;
3      for (int i = 2; 1LL * i * i <= n; ++i)
4          if (n % i == 0) {
5              res = res / i * (i - 1);
6              while (n % i == 0) n /= i;
7          }
8      if (n > 1) res = res / n * (n - 1);
9      return res;
10 }

```

1.5 扩展欧几里得 / 逆元

代码块

```

1  // 扩展gcd, 返回gcd(a,b), 并求出ax+by=gcd的一组解
2  long long exgcd(long long a, long long b, long long &x, long long &y) {
3      if (b == 0) { x = 1; y = 0; return a; }
4      long long d = exgcd(b, a % b, y, x);
5      y -= a / b * x;
6      return d;
7  }
8  // 逆元 (m为质数)
9  long long inv(long long a, long long m) { return qpow(a, m - 2, m); }

```

二、图论

2.1 存储方式

方式	空间	适用
邻接矩阵	$O(n^2)$	稠密图 / 需快速判边
邻接表	$O(n + m)$	稀疏图 / 常规遍历
边集数组	$O(m)$	Kruskal

代码块

```
1 // 邻接表
2 vector<int> g[N];
3 void add(int u, int v) { g[u].push_back(v); }
4
5 // 带权邻接表
6 vector<pair<int,int>> g[N]; // {to, weight}
```

2.2 DFS / BFS

代码块

```
1 bool vis[N];
2 void dfs(int u) {
3     vis[u] = true;
4     for (int v : g[u]) if (!vis[v]) dfs(v);
5 }
6
7 void bfs(int s) {
8     fill(dist, dist + n + 1, -1);
9     queue<int> q;
10    dist[s] = 0; q.push(s);
11    while (!q.empty()) {
12        int u = q.front(); q.pop();
13        for (int v : g[u]) if (dist[v] == -1) {
14            dist[v] = dist[u] + 1;
15            q.push(v);
16        }
17    }
18 }
```

2.3 最短路

算法	适用	复杂度

BFS	无权图	$O(n + m)$
Dijkstra+堆	非负权	$O(m \log n)$
Bellman-Ford	有负权	$O(nm)$
SPFA	有负权（平均快）	$O(nm)$ 最坏
Floyd	全源最短路	$O(n^3)$

代码块

```

1  // Dijkstra 堆优化
2  using P = pair<long long, int>;
3  void dijkstra(int s) {
4      fill(dist, dist + n + 1, INF);
5      dist[s] = 0;
6      priority_queue<P, vector<P>, greater<P>> pq;
7      pq.push({0, s});
8      while (!pq.empty()) {
9          auto [d, u] = pq.top(); pq.pop();
10         if (d != dist[u]) continue;
11         for (auto [v, w] : g[u])
12             if (dist[v] > dist[u] + w) {
13                 dist[v] = dist[u] + w;
14                 pq.push({dist[v], v});
15             }
16     }
17 }
```

2.4 最小生成树

算法	策略	复杂度
Kruskal	按边权排序 + 并查集	$O(m \log m)$
Prim	类似 Dijkstra	$O(m \log n)$

代码块

```

1  // 并查集
2  struct DSU {
3      vector<int> f;
4      DSU(int n): f(n+1) { iota(f.begin(), f.end(), 0); }
5      int find(int x) { return f[x] == x ? x : f[x] = find(f[x]); }
```

```

6     bool merge(int a, int b) {
7         a = find(a); b = find(b);
8         if (a == b) return false;
9         f[a] = b; return true;
10    }
11 };

```

2.5 拓扑排序

代码块

```

1  vector<int> g[N];
2  int indeg[N];
3  vector<int> topoSort(int n) {
4      queue<int> q;
5      for (int i = 1; i <= n; ++i) if (indeg[i] == 0) q.push(i);
6      vector<int> res;
7      while (!q.empty()) {
8          int u = q.front(); q.pop();
9          res.push_back(u);
10         for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
11     }
12     return res; // size < n 则有环
13 }

```

三、算法技巧

3.1 前缀和

代码块

```

1  // 一维
2  s[i] = s[i-1] + a[i];
3  sum(l, r) = s[r] - s[l-1];
4
5  // 二维
6  s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j];
7  sum(x1,y1,x2,y2) = s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1];

```

3.2 差分数组

```

1 // 区间 [l,r] 加 v
2 d[l] += v; d[r+1] -= v;
3 // 前缀和还原
4 for (int i = 1; i <= n; ++i) d[i] += d[i-1];

```

3.3 二分 / 二分答案

代码块

```

1 // 二分查找 lower_bound
2 int l = 0, r = n - 1, ans = -1;
3 while (l <= r) {
4     int mid = (l + r) >> 1;
5     if (a[mid] >= target) { ans = mid; r = mid - 1; }
6     else l = mid + 1;
7 }
8
9 // 二分答案模板 (最大化/最小化)
10 long long l = L, r = R, ans = 0;
11 while (l <= r) {
12     long long mid = (l + r) >> 1;
13     if (check(mid)) { ans = mid; l = mid + 1; } // 或 r = mid - 1
14     else r = mid - 1; // 或 l = mid + 1
15 }

```

3.4 双指针 / 滑动窗口

代码块

```

1 for (int l = 0, r = 0; r < n; ++r) {
2     // 扩展右边界, 加入 a[r]
3     while (!valid()) {
4         // 收缩左边界, 移除 a[l]
5         ++l;
6     }
7     // 更新答案
8 }

```

3.5 离散化

代码块

```

1 vector<int> xs(a, a + n);
2 sort(xs.begin(), xs.end());

```

```

3 xs.erase(unique(xs.begin(), xs.end()), xs.end());
4 int getId(int x) { return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1; }

```

四、动态规划

4.1 背包问题速查

类型	状态转移	一维循环方向
0/1 背包	<code>dp[j]=max(dp[j], dp[j-w]+v)</code>	逆序 $j: W \rightarrow w$
完全背包	<code>dp[j]=max(dp[j], dp[j-w]+v)</code>	正序 $j: w \rightarrow W$
多重背包（二进制拆分）	拆成 0/1 物品	逆序
分组背包	每组内部枚举选哪个	逆序

4.2 LCS / LIS

代码块

```

1 // LCS O(nm)
2 dp[i][j] = (a[i]==b[j]) ? dp[i-1][j-1]+1 : max(dp[i-1][j], dp[i][j-1]);
3
4 // LIS O(n log n)
5 vector<int> d;
6 for (int x : a) {
7     auto it = lower_bound(d.begin(), d.end(), x);
8     if (it == d.end()) d.push_back(x); else *it = x;
9 }

```

4.3 区间 DP

代码块

```

1 // 按长度从小到大枚举
2 for (int len = 2; len <= n; ++len)
3     for (int l = 1; l + len - 1 <= n; ++l) {
4         int r = l + len - 1;

```

```

5         dp[l][r] = INF;
6         for (int k = l; k < r; ++k)
7             dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r] + cost);
8     }

```

4.4 树形 DP

代码块

```

1 void dfs(int u, int fa) {
2     dp[u][0] = 0; dp[u][1] = w[u];
3     for (int v : g[u]) if (v != fa) {
4         dfs(v, u);
5         dp[u][0] += max(dp[v][0], dp[v][1]);
6         dp[u][1] += dp[v][0];
7     }
8 }

```

4.5 状态压缩 DP

代码块

```

1 // 枚举子集
2 for (int S = 0; S < (1 << n); ++S)
3     for (int sub = S; sub; sub = (sub - 1) & S)
4         // dp[S] = min/max(dp[S], dp[sub] + dp[S^sub] + ...)
5
6 // 枚举超集
7 for (int S = 0; S < (1 << n); ++S)
8     for (int sub = S; ; sub = (sub - 1) & S) { ... if (sub==0) break; }

```

五、高级数据结构

5.1 树状数组 (BIT)

代码块

```

1 struct BIT {
2     int n; vector<long long> c;
3     BIT(int n = 0) { init(n); }
4     void init(int n_) { n = n_; c.assign(n + 1, 0); }
5     void add(int x, long long v) { for (; x <= n; x += x & -x) c[x] += v; }

```



```

6      long long sum(int x) const { long long r = 0; for (; x > 0; x -= x & -x) r
    += c[x]; return r; }
7      long long rangeSum(int l, int r) const { return sum(r) - sum(l - 1); }
8  };

```

5.2 线段树（区间加 + 区间和）

代码块

```

1  struct SegTree {
2      int n;
3      vector<long long> sum, lazy;
4      SegTree(int n = 0) { init(n); }
5      void init(int n_) { n = n_; sum.assign(4*n+4,0); lazy.assign(4*n+4,0); }
6      void apply(int o, int l, int r, long long v) { sum[o] += (r-l+1)*v;
    lazy[o] += v; }
7      void push(int o, int l, int r) {
8          if (!lazy[o]) return;
9          int mid = (l+r)>>1;
10         apply(o<<1, l, mid, lazy[o]);
11         apply(o<<1|1, mid+1, r, lazy[o]);
12         lazy[o] = 0;
13     }
14     void update(int o, int l, int r, int L, int R, long long v) {
15         if (L <= l && r <= R) { apply(o,l,r,v); return; }
16         push(o,l,r);
17         int mid = (l+r)>>1;
18         if (L <= mid) update(o<<1,l,mid,L,R,v);
19         if (R > mid) update(o<<1|1,mid+1,r,L,R,v);
20         sum[o] = sum[o<<1] + sum[o<<1|1];
21     }
22     long long query(int o, int l, int r, int L, int R) {
23         if (L <= l && r <= R) return sum[o];
24         push(o,l,r);
25         int mid = (l+r)>>1; long long res = 0;
26         if (L <= mid) res += query(o<<1,l,mid,L,R);
27         if (R > mid) res += query(o<<1|1,mid+1,r,L,R);
28         return res;
29     }
30 };

```

5.3 字符串哈希

代码块

```

1  using ull = unsigned long long;
2  const ull BASE = 131;
3  ull h[N], p[N];
4  void init(const string &s) {
5      p[0] = 1;
6      for (int i = 1; i <= s.size(); ++i) {
7          h[i] = h[i-1] * BASE + s[i-1];
8          p[i] = p[i-1] * BASE;
9      }
10 }
11 ull getHash(int l, int r) { return h[r] - h[l-1] * p[r-l+1]; } // 1-based, 自然
    溢出

```

5.4 KMP

代码块

```

1  vector<int> prefix_function(const string &s) {
2      int n = s.size();
3      vector<int> pi(n);
4      for (int i = 1; i < n; ++i) {
5          int j = pi[i-1];
6          while (j > 0 && s[i] != s[j]) j = pi[j-1];
7          if (s[i] == s[j]) ++j;
8          pi[i] = j;
9      }
10     return pi;
11 }

```

5.5 Tarjan

代码块

```

1  int dfn[N], low[N], tim, stk[N], top, inStk[N], scc[N], sccCnt;
2  void tarjan(int u) {
3      dfn[u] = low[u] = ++tim;
4      stk[++top] = u; inStk[u] = 1;
5      for (int v : g[u]) {
6          if (!dfn[v]) { tarjan(v); low[u] = min(low[u], low[v]); }
7          else if (inStk[v]) low[u] = min(low[u], dfn[v]);
8      }
9      if (dfn[u] == low[u]) {
10         ++sccCnt;
11         while (true) {
12             int x = stk[top--];

```

```
13         inStk[x] = 0; scc[x] = sccCnt;
14         if (x == u) break;
15     }
16 }
17 }
```

六、复杂度速查表

算法/数据结构	时间	空间
快速幂	$O(\log b)$	$O(1)$
欧拉筛	$O(n)$	$O(n)$
DFS/BFS	$O(n + m)$	$O(n + m)$
Dijkstra+堆	$O(m \log n)$	$O(n + m)$
Floyd	$O(n^3)$	$O(n^2)$
Kruskal	$O(m \log m)$	$O(n + m)$
前缀和查询	$O(1)$	$O(n)$
树状数组	$O(\log n)$ / 操作	$O(n)$
线段树	$O(\log n)$ / 操作	$O(4n)$
KMP	$O(n + m)$	$O(m)$
0/1 背包	$O(nW)$	$O(W)$
LCS	$O(nm)$	$O(nm)$
区间 DP	$O(n^3)$	$O(n^2)$
状态压缩 DP	$O(n \cdot 2^n)$	$O(2^n)$

七、头文件速查

代码块

```
1 #include <bits/stdc++.h> // 万能头 (竞赛常用)
```

```
2  #include <numeric>           // gcd, lcm (C++17)
3  #include <algorithm>         // sort, lower_bound, unique...
4  #include <queue>             // queue, priority_queue
5  #include <stack>             // stack
6  #include <vector>            // vector
7  #include <map>               // map
8  #include <set>               // set
9  #include <cstring>           // memset
10 #include <cmath>             // sqrt, pow, log...
```